README.md

GEN AI-POWERED CUSTOMER RISK RATING ENGINE (V2 — REAL TRAINED PD MODEL)

This is the step up from the Colab prototype: the hand-written PD formula is
replaced with an **actual trained gradient boosting model**, plus a fairness
testing scaffold that was entirely missing before.

FILES

File	Purpose
`data/generate_synthetic_data.py`	Synthetic loan dataset (stand-in for real data — see below)
`train_pd_model.py`	Trains the PD model, saves it, prints AUC/Brier score, computes PD deciles
`pd_model.py`	Loads the trained model and scores new applicants
`schemas.py`	Data contracts (now includes `loan_amount_requested`, `open_trade_lines`, `credit_utilization` — the trained model's full feature set — plus `applicant_group` for fairness testing only)
`document_intelligence.py`	Document extraction layer (Gemini)
`risk_scoring.py`	Gen AI reasoning/scoring layer (Gemini), now anchored to the PD decile
`workflow.py`	Deterministic score→tier→action routing + audit logging
`fairness_check.py`	Four-fifths rule disparate-impact screen on the audit log
`policy_docs/*.md`	Underwriting policy knowledge base (RAG source documents)
`rag/build_index.py`	Chunks and embeds the policy docs into a persistent Chroma vector database
`rag/retriever.py`	Embeds an applicant query and retrieves the most relevant policy chunks via Chroma
`app.py`	Streamlit frontend -- form input, results display, audit log viewer, fairness check viewer
`main.py`	End-to-end orchestration (also used by app.py)

SETUP

```bash
pip install -r requirements.txt
export GEMINI_API_KEY="your-gemini-api-key-here"   # free at aistudio.google.com/apikey

1. TRAIN THE PD MODEL (CREATES MODEL_ARTIFACTS/PD_MODEL.JOBLIB)

python train_pd_model.py

2. BUILD THE RAG POLICY INDEX (EMBEDS POLICY_DOCS/*.MD -- RUN ONCE, AND

AGAIN WHENEVER YOU EDIT/ADD A POLICY DOCUMENT)

python rag/build_index.py

3. RUN A SAMPLE APPLICANT THROUGH THE FULL PIPELINE

python main.py

4. RUN THE FAIRNESS SCREEN ON WHATEVER'S IN THE AUDIT LOG

python fairness_check.py
```

RAG LAYER (RETRIEVAL-AUGMENTED GENERATION)

PRESENTATION_GUIDE.md

CLIENT PRESENTATION GUIDE — GEN AI PERSONAL LOAN RISK RATING ENGINE

This document prepares you to present the risk-rating engine to clients, stakeholders, or technical reviewers. It includes an elevator pitch, slide outline, demo script, talking points, technical appendix, deployment checklist, and FAQs.

ELEVATOR PITCH (30–60S)

We built a prototype end-to-end personal loan risk-rating engine that combines a trained machine-learning PD model with an LLM-based, policy-grounded reasoning layer. The system computes a probability of default, maps it to deciles and a consumer-facing 1–10 risk score, and produces a grounded explanation citing the exact underwriting policy passages that influenced the decision. It includes audit logging and a four-fifths preliminary fairness screen to support governance and human review.

SLIDE DECK OUTLINE (10–12 SLIDES)

1. Title & Team — Project name, presenters, date.
2. Problem & Opportunity — Why better risk scoring matters today (accuracy, speed, fairness, auditability).
3. Solution Overview — One-slide architecture: ML model + RAG with Gemini + Chroma + Streamlit UI + SQLite audit log.
4. Key Capabilities — Scoring, RAG-grounded explanations, audit log, fairness screen, human override.
5. Demo Snapshot — Screenshot or short clip of Streamlit UI.
6. Technical Stack — Python, scikit-learn, pandas, numpy, joblib, pydantic, google-genai (Gemini), chromadb, streamlit, SQLite.
7. Data & Model — Training dataset, features, PD decile mapping, evaluation metrics (AUC, Brier).
8. Governance & Controls — Audit log structure, four-fifths screen, human-in-the-loop flow.
9. Security & Privacy — Secrets management, no PII in logs (recommendations), deployment notes.
10. Limitations & Risks — Synthetic-data caveat, model drift, legal review required for production.
11. Roadmap & Next Steps — Retrain on real data, CI/CD, monitoring, integration with core systems.
12. Q&A / Appendix pointer — Link to repo, demo steps, README.

DEMO SCRIPT (5–8 MINUTES)

Goal: Show the full flow end-to-end while emphasizing governance and explainability.

Preparation (1–2 minutes):
- Ensure `GEMINI_API_KEY` is set in the environment or in `.streamlit/secrets.toml`.
- Run `pip install -r requirements.txt`.
- (Optional) Train or load the shipped model: `python train_pd_model.py` (or ensure `model_artifacts/pd_model.joblib` exists).
- Build RAG index (if you modified `policy_docs/`): `python rag/build_index.py`.
- Start the UI: `streamlit run app.py` and open `http://localhost:8501` (or the port displayed).

Demo steps:
1. Open the "Score an Applicant" tab.
2. Paste a short bank statement into the document field and fill structured fields (use an example from `README.md`).

TECHNICAL_DEEP_DIVE.md

TECHNICAL DEEP DIVE — GEN AI PERSONAL LOAN RISK RATING ENGINE

This deep-dive documents architecture, data flow, model training, RAG/LLM design, governance, deployment, and maintainability considerations.

1. HIGH-LEVEL ARCHITECTURE

- Ingest: structured applicant fields + free-text documents (bank statements). Document extraction produces structured facts.
- Model: trained PD model (`GradientBoostingClassifier`) produces PD probability. Model persisted to `model_artifacts/pd_model.joblib`.
- Decile mapping: PDs mapped to deciles (1-10) against the training distribution to stabilize downstream reasoning.
- RAG: Gemini embeddings via `google-genai` produce vectors for policy document chunks. Chroma (`chromadb`) stores the vector index in `rag_index/chroma_db/`.
- LLM reasoning: `risk_scoring.py` constructs a prompt that injects retrieved policy chunks and asks Gemini to produce a risk assessment, top contributing factors, customer-facing explanation, and policy sources cited.
- Orchestration: `main.py` and `workflow.py` glue document extraction, model scoring, RAG retrieval, LLM call, routing to tier/actions, and audit logging.
- UI: `app.py` (Streamlit) exposes scoring, audit viewer, fairness check, and human override flows.
- Persistence: `db.py` stores audit records in SQLite by default (`risk_engine.db`).

2. CODE MAP (KEY FILES)

- `train_pd_model.py` — training pipeline, synthetic-data generator, model evaluation, saves `pd_model.joblib` and `pd_decile_to_score.joblib`.
- `pd_model.py` — model loading and scoring helper; computes PD and decile mapping.
- `data/generate_synthetic_data.py` — synthetic data generator and feature definitions.
- `document_intelligence.py` — document extraction layer; converts raw text into structured observations (nsf events, observed income, anomalies).
- `risk_scoring.py` — builds the LLM prompt, calls Gemini (via `gemini_client.py`), parses responses.
- `gemini_client.py` — thin client wrapper around `google-genai` calls (embeddings + LLM).
- `rag/build_index.py` — chunks `policy_docs/*.md`, embeds chunks, upserts into Chroma vector DB (`rag_index/`).
- `rag/retriever.py` — given applicant context, embeds query and retrieves top-K policy chunks.
- `workflow.py` — deterministic routing from LLM assessment to `score -> tier -> action`, and audit logging calls to `db.py`.
- `db.py` — SQLite schema and helper functions: `log_audit_record()`, `load_audit_log()`, `record_human_override()`.
- `fairness_check.py` — four-fifths rule screen implemented over the audit log.
- `schemas.py` — Pydantic models for structured inputs/outputs.
- `app.py` — Streamlit frontend wiring UI to the pipeline.

3. DATA MODEL & FEATURES

Feature set expected by `train_pd_model.py` / `pd_model.py`:
- `bureau_score` (int)
- `debt_to_income` (float)
- `nsf_events_3m` (int)
- `employment_tenure_months` (int)
- `stated_monthly_income` (float)
- `loan_amount_requested` (float)
- `open_trade_lines` (int)
- `credit_utilization` (float)

EXECUTIVE_SUMMARY.md

Gen AI Personal Loan Risk Rating Engine — Executive Summary

Purpose

A prototype end-to-end risk-rating engine that combines a trained probability-of-default (PD) model with an LLM-powered, policy-grounded reasoning layer. The system produces numeric PDs, maps them to decile-based risk scores, and generates grounded, auditable explanations tied to the bank's underwriting policies.

Value Proposition

- Faster, more consistent decisions: automated scoring and policy-grounded explanations reduce manual review workload.
- Explainability & auditability: each decision includes the PD, decile, routed action, and the exact policy excerpts cited by the LLM (RAG), plus an audit record for governance.
- Governance-ready controls: audit logs, human-overrides, and a four-fifths screening scaffold support monitoring and compliance workflows.

Key Capabilities

- Trained ML model for PD estimation (scikit-learn, persisted via joblib).
- RAG layer using Google Gemini embeddings and Chroma vector store to ground LLM explanations in policy documents.
- Streamlit-based demo UI for scoring, audit review, and fairness checks.
- Lightweight governance: SQLite-backed audit log and human-override recording.
- Schema validation with Pydantic and a deterministic routing/workflow layer.

Business Benefits

- Short-term: rapid prototype for stakeholder demos, policy validation, and human-in-the-loop pilots.
- Medium-term: reduce underwriter time per file and increase consistency of routing decisions.
- Long-term: feed real performance data to productionize the PD model, enable automated monitoring and CI of model performance and fairness.

Limitations & Risks

- Prototype status: model is trained on synthetic data — retraining on institution data is required before production use.
- Regulatory & legal: the four-fifths screen is a preliminary check only; formal legal and statistical review required for compliance.
- Data & extraction: document extraction has not been validated on real messy documents; accuracy must be measured before production rollout.

Deployment Readiness & Requirements

- Secrets management: secure storage of Gemini API key (supported via `.streamlit/secrets.toml` for demos; use a secrets manager in production).
- Persistence: replace the bundled SQLite with a managed DB for production (Postgres, Neon, Supabase).
- Monitoring: add model performance and drift monitoring, logging retention and access controls.

Recommended Next Steps

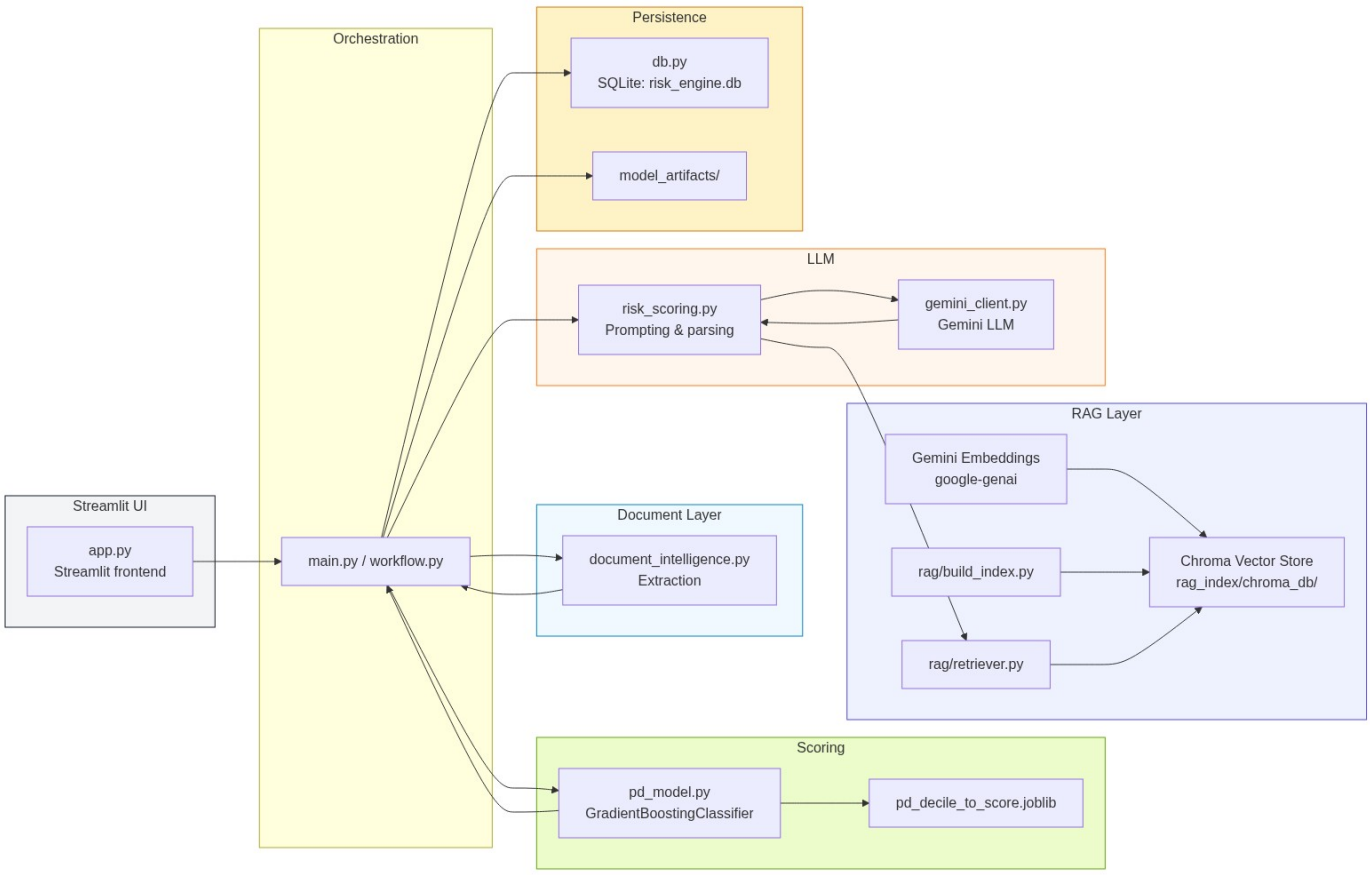
1. Retrain PD model on historical loan performance; validate with holdout tests and business KPIs (AUC, Brier).
2. Conduct legal and compliance review, and perform rigorous fairness testing beyond the four-fifths screen.
3. Replace SQLite with managed DB and deploy under access-controlled environment.
4. Add CI tests, monitoring dashboards, and alerting for model drift and policy changes.

ARCHITECTURE (Mermaid source)

```
%% Mermaid architecture diagram for Gen AI Personal Loan Risk Rating Engine
%% Render with a Mermaid viewer or in Markdown that supports Mermaid.

flowchart LR
    subgraph UI[Streamlit UI]
        A[app.py<br>Streamlit frontend]
    end

    subgraph Orchestration[Orchestration]
        B[main.py / workflow.py]
    end

    subgraph Scoring[Scoring]
        C[pd_model.py<br>GradientBoostingClassifier]
        D[pd_decile_to_score.joblib]
    end

    subgraph Docs[Document Layer]
        E[document_intelligence.py<br>Extraction]
    end

    subgraph RAG[RAG Layer]
        F[rag/retriever.py]
        G[rag/build_index.py]
        H[Chroma Vector Store<br/>rag_index/chroma_db/]
        I[Gemini Embeddings<br>google-genai]
    end

    subgraph LLM[LLM]
        J[gemini_client.py<br>Gemini LLM]
        K[risk_scoring.py<br>Prompting & parsing]
    end

    subgraph Persistence[Persistence]
        L[db.py<br>SQLite: risk_engine.db]
        M[model_artifacts/]
    end

    A --> B
    B --> C
    B --> E
    E --> B
    C --> B
    C --> D

    B --> K
    K --> J
    K --> F
    I --> H
    G --> H
    F --> H
    J --> K

    B --> L
    B --> M

    style UI fill:#f3f4f6,stroke:#111827
    style RAG fill:#eef2ff,stroke:#4f46e5
    style LLM fill:#fff7ed,stroke:#ff7a18
    style Scoring fill:#ecfccb,stroke:#65a30d
    style Persistence fill:#fef3c7,stroke:#d97706
    style Docs fill:#f0f9ff,stroke:#0284c7

    click H "./rag_index/" "Open rag_index/"
    click M "./model_artifacts/" "Open model_artifacts/"
```

ADDITIONAL ARTIFACTS GENERATED IN REPO

- EXECUTIVE_SUMMARY.pdf
- ARCHITECTURE_SLIDE.pptx
- ARCHITECTURE.mmd


Persistence
db.py
SQLite: risk_engine.db
model_artifacts/
Orchestration
LLM
risk_scoring.py
Prompting & parsing
gemini_client.py
Gemini LLM
RAG Layer
Gemini Embeddings
google-genai
Streamlit UI
app.py
Streamlit frontend
main.py / workflow.py
Document Layer
document_intelligence.py
Extraction
rag/build_index.py
Chroma Vector Store
rag_index/chroma_db/
rag/retriever.py
Scoring
pd_model.py
GradientBoostingClassifier
pd_decile_to_score.joblib